GENAI ANALYSIS REPORT

EECS 348 | Assignment 6: Sudoku Solver AI Code Refactoring

A & B. GENAI PROGRAMS & ACCESS METHODS

GENAI PROGRAM	DEVELOPER	ACCESS METHOD
Gemini	Google	Web-based interactive chat interface (gemini.google.com)
ChatGPT	OpenAI	Web-based interactive chat interface (chatgpt.com)

C. PROMPT USED

> *"Modify the provided C++ Sudoku solver code (Assignment 5) to write the solutions for each puzzle file to a single file named solutions.txt. For each puzzle, include a single line: 'Puzzle: x', where x is the filename. If no solutions are found, output 'No solution found.'. If solutions are found, output all solutions followed by 'Solutions found: n', where n is the number of solutions. The solutions should be in the same format as the input files. Do not replace the entire file with your code; only make the necessary modifications. Add 'Revised by: [GenAI Name]' and the revision date to the Profile Comments."*

E. COMPARATIVE TECHNICAL ANALYSIS

CORRECTNESS	Both GenAIs met the primary requirements by redirecting file output to a single shared `solutions.txt` stream with required headers (`"Puzzle: x"`) and footers (`"Solutions found: n"`). ChatGPT provided slightly stricter stream validation by checking `outputFile.good()` inside `solve()` and checking `!outFile` post-flush in `main()`.
EXECUTION TIME	Both utilize recursive backtracking with $O(9^K)$ worst-case complexity. Gemini maintained an MRV optimization early exit (`fewestCandidates == 1`) to prevent unnecessary array scans. ChatGPT omitted this early check, performing full 81-cell iterations per search node. Gemini executes slightly faster per node.
SPACE COMPLEXITY	Identical and optimal across both models at $O(1)$ auxiliary space. Grid memory relies on a fixed 9×9 integer array, and recursion stack depth is capped at 81 frames.
MAINTAINABILITY	Gemini preserved all original line-by-line comments from Assignment 5 and added clear `// CHANGED:` annotations. ChatGPT removed almost all inline comments, which severely violates the course documentation rubric requiring line-by-line comments for full credit.

F. SELECTION & JUSTIFICATION

Selected AI: **Gemini (Google)**

Justification: Gemini was selected due to documentation compliance. The grading rubric explicitly requires "comments on every line" for full credit. ChatGPT removed almost all existing line comments, while Gemini preserved them and clearly marked all new modifications with `// CHANGED:` tags.

G. APPLIED IMPROVEMENTS

1. **Correctness:** Integrated ChatGPT's rigorous stream state verification (`outputFile.good()` checks and post-close error trapping) into Gemini's code.
2. **Execution Time:** Retained Gemini's early-exit candidate search optimization (`fewestCandidates == 1`) for reduced node traversal time.
3. **Space Complexity:** Maintained optimal fixed $O(1)$ memory allocation.
4. **Maintainability:** Preserved line-by-line comments across all modified methods and updated the prologue comments to credit both GenAI tools.

D. GENERATED CODE — GEMINI (GOOGLE)

Complete C++ Implementation | Assignment 6 Revision

GEMINI GENERATED CODE

```
/*
* Program: EECS 348 Assignment 6 (revision of Assignment 5)
* Description: Loads five Sudoku puzzles, finds every solution using recursive
*              backtracking, displays results, and saves all solutions to a single solutions.txt file.
* Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
* Outputs: Console results and solutions.txt
* Original author: Prabesh Sapkota
* Original creation date: October 15, 2026
* Revised by: Gemini
* Revision date: November 12, 2026
* Collaborators: Original Assignment 4/5 authors
* Other sources: Modified via AI prompt as per Assignment 6 guidelines.
* Revision notes: Changed file output logic to write all puzzle solutions to a single "solutions.txt" file.
*/

#include <iostream> // Provides console input and output.
#include <fstream> // Provides file-reading tools.
#include <string> // Provides the string type.
#include <vector> // Stores and validates the puzzle's 81 input cells.
#include <cctype> // Provides the isspace function.
using namespace std; // Allows standard-library names without the std:: prefix.

class SudokuSolver { // Defines a class that stores and solves one Sudoku puzzle.
private: // Begins the private section of the class.
    int board[9][9]; // Stores the Sudoku grid; zero represents an empty cell.
    int solutionCount; // Counts how many complete solutions have been found.
    // CHANGED: Removed ofstream outputFile from class members since we now pass a single shared stream.

    bool isSafe(int row, int col, int digit) { // Checks whether digit can go in the specified cell.
        for (int j = 0; j < 9; j++) { // Checks all cells in the selected row.
```

```
        if (board[row][j] == digit) { // Detects the same digit in that row.
            return false; // Rejects a digit that already appears in the row.
        } // Ends the row-conflict condition.
    } // Ends the row scan.

    for (int i = 0; i < 9; i++) { // Checks all cells in the selected column.
        if (board[i][col] == digit) { // Detects the same digit in that column.
            return false; // Rejects a digit that already appears in the column.
        } // Ends the column-conflict condition.
    } // Ends the column scan.

    int startRow = (row / 3) * 3; // Finds the first row of the cell's 3-by-3 block.
    int startCol = (col / 3) * 3; // Finds the first column of the cell's 3-by-3 block.

    for (int i = startRow; i < startRow + 3; i++) { // Visits the three rows of the block.
        for (int j = startCol; j < startCol + 3; j++) { // Visits the three columns of the block.
            if (board[i][j] == digit) { // Detects the same digit inside the block.
                return false; // Rejects a digit that already appears in the block.
            } // Ends the block-conflict condition.
        } // Ends the inner block scan.
    } // Ends the outer block scan.

    return true; // Confirms that the digit follows all Sudoku placement rules.
} // Ends isSafe.

void printBoard() const { // Prints the current Sudoku grid without modifying it.
    for (int i = 0; i < 9; i++) { // Visits each row of the board.
        if (i % 3 == 0) { // Checks whether a new group of three rows begins.
            cout << "+-------+-------+-------+" << endl; // Prints the horizontal grid border.
        } // Ends the row-group condition.
```

```
        for (int j = 0; j < 9; j++) { // Visits each cell in the current row.
            if (j % 3 == 0) { // Checks whether a new group of three columns begins.
                cout << "| "; // Prints the left boundary of a 3-column group.
            } // Ends the column-group condition.

            if (board[i][j] == 0) { // Checks whether the current cell is empty.
                cout << "_ "; // Displays an underscore for an empty cell.
            } else { // Handles a cell containing a digit.
                cout << board[i][j] << " "; // Displays the digit stored in the cell.
            } // Ends the empty/non-empty condition.
        } // Ends the scan of the current row.

        cout << "|" << endl; // Prints the right boundary and starts a new output line.
    } // Ends the scan of all rows.

    cout << "+-------+-------+-------+" << endl; // Prints the bottom border of the grid.
} // Ends printBoard.

void writeBoardToFile(ofstream& outputFile) { // CHANGED: Added outputFile reference parameter.
    for (int i = 0; i < 9; i++) { // Visits each row of the board.
        for (int j = 0; j < 9; j++) { // Visits each cell in the current row.
            if (j > 0) { // Checks whether a separator is needed before this cell.
                outputFile << " "; // Separates cells with a single space, as in the input files.
            } // Ends the separator condition.

            if (board[i][j] == 0) { // Checks whether the current cell is empty.
                outputFile << "_"; // Writes an underscore for an empty cell, matching the input format.
            } else { // Handles a cell containing a digit.
                outputFile << board[i][j]; // Writes the digit stored in the cell.
```

```
            } // Ends the empty/non-empty condition.
        } // Ends the scan of the current row.

        outputFile << "\n"; // Ends the current row of the output file.
    } // Ends the scan of all rows.
} // Ends writeBoardToFile.

bool findBestEmptyCell(int& bestRow, int& bestCol, unsigned int& bestMask) { // Unchanged from Assignment
5
    int fewestCandidates = 10; // No cell can have more than nine options.
    bestRow = -1; // -1 means we have not found an empty cell yet.
    bestCol = -1; // Initializes best column tracker.
    bestMask = 0; // Initializes the bitmask for valid digits.

    for (int row = 0; row < 9; row++) { // Search all rows.
        for (int col = 0; col < 9; col++) { // Search each cell in the row.
            if (board[row][col] != 0) { // Ignore digits that are already fixed.
                continue; // Moves to the next cell.
            } // Ends fixed digit check.

            int candidateCount = 0; // Tracks number of legal digits for this cell.
            unsigned int candidateMask = 0; // Each bit represents a legal digit.
            for (int digit = 1; digit <= 9; digit++) { // Tests digits 1 through 9.
                if (isSafe(row, col, digit)) { // Use the original rule checker.
                    candidateCount++; // Increments the legal candidate count.
                    candidateMask |= (1U << digit); // Remember this legal option.
                } // Ends safety check.
            } // Ends digit loop.

            if (candidateCount == 0) { // This branch cannot lead to a solution.
                bestRow = row; // Records the dead-end row.
```

```
                bestCol = col; // Records the dead-end column.
                bestMask = 0; // Indicates no valid candidates.
                return true; // Forces early termination of this path.
            } // Ends dead-end check.

            if (candidateCount < fewestCandidates) { // Use the most restricted cell.
                fewestCandidates = candidateCount; // Updates the minimum candidates found.
                bestRow = row; // Records the best row.
                bestCol = col; // Records the best column.
                bestMask = candidateMask; // Save its legal digits for recursion.
                if (fewestCandidates == 1) { // Checks if we found a cell with exactly one option.
                    return true; // We cannot improve on one legal choice.
                } // Ends single-candidate check.
            } // Ends optimization check.
        } // Ends column loop.
    } // Ends row loop.
    return bestRow != -1; // False only when all cells have been filled.
} // Ends findBestEmptyCell.

 void solveRecursive(ofstream& outputFile) { // CHANGED: Passed the shared output stream for writing solutions.
    int emptyRow = -1; // Tracks the chosen empty cell row.
    int emptyCol = -1; // Tracks the chosen empty cell column.
    unsigned int candidates = 0; // Stores the legal digit mask for the cell.

    if (!findBestEmptyCell(emptyRow, emptyCol, candidates)) { // Puzzle is complete.
        solutionCount++; // Increments total solutions found.
        cout << "\nSolution " << solutionCount << ":" << endl; // Prints to console.
        printBoard(); // Preserve Assignment 4's console display.
        writeBoardToFile(outputFile); // CHANGED: Saves this solution using the shared stream.
        return; // Keep backtracking to find any other solutions.
```

```
        } // Ends completion check.

        if (candidates == 0) { // No digit fits the chosen cell.
            return; // Stop this unsuccessful branch immediately.
        } // Ends dead-end check.

        for (int digit = 1; digit <= 9; digit++) { // Attempts to place digits 1 through 9.
            if ((candidates & (1U << digit)) == 0) { // Skip forbidden digits.
                continue; // Moves to the next digit.
            } // Ends invalid digit check.
            board[emptyRow][emptyCol] = digit; // Try a saved legal digit.
            solveRecursive(outputFile); // CHANGED: Passes output stream down the recursion tree.
            board[emptyRow][emptyCol] = 0; // Undo the move before trying another.
        } // Ends digit loop.
    } // Ends solveRecursive.

public: // Begins the public section of the class.
    SudokuSolver() : solutionCount(0) { // Initializes the solution counter when an object is created.
        for (int row = 0; row < 9; row++) { // Visits each row during initialization.
            for (int col = 0; col < 9; col++) { // Visits each column during initialization.
                board[row][col] = 0; // Marks every cell as empty.
            } // Ends the initialization of one row.
        } // Ends board initialization.
    } // Ends the constructor.

    bool loadPuzzle(const string& filename) { // Loads and validates the basic format of a puzzle file.
        ifstream inputFile(filename); // Opens the specified file for reading.

        if (!inputFile.is_open()) { // Checks whether the file opened successfully.
            cerr << "Error: Could not open " << filename << endl; // Reports a missing or inaccessible file.
```

```
    return false; // Stops loading when the file cannot be opened.
  } // Ends the file-open check.

  vector<char> cells; // Temporarily stores all 81 cell symbols before changing the board.
  char ch; // Stores each character read from the file.

  while (inputFile.get(ch)) { // Reads the file one character at a time.
    if ((ch >= '1' && ch <= '9') || ch == '_') { // Accepts digits one through nine and underscores.
      cells.push_back(ch); // Saves a valid Sudoku cell symbol.
    } else if (ch == '0') { // Allows zero as another way to mark an empty cell.
      cells.push_back('_'); // Normalizes zero to the underscore blank marker.
    } else if (!isspace(static_cast<unsigned char>(ch))) { // Rejects unexpected non-whitespace characters.
      cerr << "Error: Invalid character in " << filename << endl; // Reports an unexpected character.
      return false; // Stops loading a malformed puzzle file.
    } // Ends the character-validation branches.
  } // Ends the file-reading loop.

  if (cells.size() != 81) { // Requires exactly 81 cell symbols for a 9-by-9 puzzle.
    cerr << "Error: " << filename << " must contain exactly 81 Sudoku cells." << endl; // Reports the incorrect cell count.
    return false; // Rejects a puzzle with too few or too many cells.
  } // Ends the cell-count check.

  for (int row = 0; row < 9; row++) { // Copies the validated symbols into each board row.
    for (int col = 0; col < 9; col++) { // Copies the symbols into each board column.
      char cell = cells[row * 9 + col]; // Retrieves the symbol at the current linear position.

      if (cell == '_') { // Checks whether the symbol represents a blank cell.
        board[row][col] = 0; // Stores blank cells internally as zero.
      } else { // Handles a symbol containing a digit.
        board[row][col] = cell - '0'; // Converts the digit character into its integer value.
```

```
            } // Ends the cell-conversion condition.
        } // Ends copying one row.
    } // Ends copying all 81 cells.

    return true; // Confirms that the file was read and formatted correctly.
} // Ends loadPuzzle.

bool isInitialBoardValid() { // Checks that the puzzle's original digits do not conflict.
    for (int row = 0; row < 9; row++) { // Examines each row of the starting puzzle.
        for (int col = 0; col < 9; col++) { // Examines each cell in the current row.
            if (board[row][col] == 0) { // Checks whether the current cell is blank.
                continue; // Skips blank cells because they cannot conflict with a given digit.
            } // Ends the blank-cell condition.

            int digit = board[row][col]; // Saves the digit currently being checked.
            board[row][col] = 0; // Temporarily clears the cell so it does not conflict with itself.
            bool valid = isSafe(row, col, digit); // Checks the saved digit against its row, column, and block.
            board[row][col] = digit; // Restores the original digit after checking it.

            if (!valid) { // Checks whether another given digit conflicts with this one.
                return false; // Rejects the starting puzzle if a conflict is found.
            } // Ends the conflict condition.
        } // Ends the scan of the current row.
    } // Ends the scan of the starting board.

    return true; // Confirms that all given digits are mutually consistent.
} // Ends isInitialBoardValid.

bool solve(const string& filename, ofstream& outputFile) { // CHANGED: Passed the single output stream by reference.
    cout << "\n========================================" << endl; // Prints console header.
```

```
        cout << "Puzzle file: " << filename << endl; // Prints filename to console.
        cout << "========================================" << endl; // Prints console footer.
        cout << "\nOriginal puzzle:" << endl; // Console log for original board.
        printBoard(); // Keep the original console presentation.

        outputFile << "Puzzle: " << filename << "\n"; // CHANGED: Added the required "Puzzle: x" header to solutions.txt.

        if (!isInitialBoardValid()) { // Don't search puzzles with conflicting clues.
            cout << "\nNo solution found: the original puzzle has conflicting digits." << endl; // Prints console failure.
            outputFile << "No solution found.\n"; // CHANGED: Writes exact required failure message directly to the stream.
            return true; // CHANGED: Return true since the puzzle was processed correctly, even if unsolved.
        } // Ends valid initial board check.

        solutionCount = 0; // Start each puzzle with a new solution count.

        solveRecursive(outputFile); // CHANGED: Passed the output stream to the recursive solver.

        if (solutionCount == 0) { // Checks if the recursive search found zero solutions.
            cout << "\nNo solution found." << endl; // Prints console failure.
            outputFile << "No solution found.\n"; // CHANGED: Writes required missing solution format.
        } else { // Handles case where at least one solution was found.
            cout << "\nTotal solutions found: " << solutionCount << endl; // Prints total to console.
            outputFile << "Solutions found: " << solutionCount << "\n"; // CHANGED: Appends the required trailing solution count string.
        } // Ends solution count check.

        return true; // CHANGED: Stream closing is handled in main, simply return success here.
    } // Ends solve.
}; // Ends the SudokuSolver class definition.
```

```
int main() { // Processes all five required puzzle files.
    const string filenames[] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"}; // Stores input filenames.
    bool allFilesProcessed = true; // Track errors while still checking all puzzles.

    ofstream outFile("solutions.txt"); // CHANGED: Open a single solutions file once for all puzzles, per Assignment 6 instructions.
    if (!outFile.is_open()) { // CHANGED: Checks if the master output file opened correctly.
        cerr << "Error: Could not create solutions.txt" << endl; // CHANGED: Error message for master file.
        return 1; // CHANGED: Exit early if we can't write to the output file.
    } // CHANGED: Ends the master file check.

    for (int i = 0; i < 5; i++) { // Loops through the 5 puzzle files.
        const string& filename = filenames[i]; // Retrieves the current filename.
        SudokuSolver solver; // Keep each puzzle's board and count separate.

        if (solver.loadPuzzle(filename)) { // Attempts to load the board into memory.
            if (!solver.solve(filename, outFile)) { // CHANGED: Passed the shared outFile stream to the solver.
                allFilesProcessed = false; // Records if the solving process failed internally.
            } // Ends solving check.
        } else { // Handles input files that could not be read or parsed.
            outFile << "Puzzle: " << filename << "\nNo solution found.\n"; // CHANGED: Writes failure string to shared file for invalid inputs.
            allFilesProcessed = false; // Marks failure in processing track.
        } // Ends load condition check.
    } // Ends puzzle loop.

    outFile.close(); // CHANGED: Closes the master output file once all puzzles are processed.
    return allFilesProcessed ? 0 : 1; // Nonzero means at least one file failed.
} // Ends main.
```

```
/*
* Program: EECS 348 Assignment 6 (revision of Assignment 5)
* Description: Loads five Sudoku puzzles, finds every solution using recursive
*              backtracking, displays results, and saves all solutions to a single solutions.txt file.
* Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
* Outputs: Console results and solutions.txt
* Original author: Prabesh Sapkota
* Original creation date: October 15, 2026
* Revised by: Gemini
* Revision date: November 12, 2026
* Collaborators: Original Assignment 4/5 authors
* Other sources: Modified via AI prompt as per Assignment 6 guidelines.
* Revision notes: Changed file output logic to write all puzzle solutions to a single "solutions.txt" file.
*/
```

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cctype>
using namespace std;

class SudokuSolver {
private:
    int board[9][9];
    int solutionCount;

    bool isSafe(int row, int col, int digit) {
        for (int j = 0; j < 9; j++) {
            if (board[row][j] == digit) {
                return false;
            }
        }

        for (int i = 0; i < 9; i++) {
            if (board[i][col] == digit) {
                return false;
            }
        }

        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        for (int i = startRow; i < startRow + 3; i++) {
            for (int j = startCol; j < startCol + 3; j++) {
```

```
            if (board[i][j] == digit) {
                return false;
            }
        }
    }

    return true;
}

void printBoard() const {
    for (int i = 0; i < 9; i++) {
        if (i % 3 == 0) {
            cout << "+-------+-------+-------+" << endl;
        }

        for (int j = 0; j < 9; j++) {
            if (j % 3 == 0) {
                cout << "| ";
            }

            if (board[i][j] == 0) {
                cout << "_ ";
            } else {
                cout << board[i][j] << " ";
            }
        }

        cout << "|" << endl;
    }
```

```
    cout << "+-------+-------+-------+" << endl;
}

void writeBoardToFile(ofstream& outputFile) {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (j > 0) {
                outputFile << " ";
            }

            if (board[i][j] == 0) {
                outputFile << "_";
            } else {
                outputFile << board[i][j];
            }
        }

        outputFile << "\n";
    }
}

bool findBestEmptyCell(int& bestRow, int& bestCol, unsigned int& bestMask) {
    int fewestCandidates = 10;
    bestRow = -1;
    bestCol = -1;
    bestMask = 0;

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            if (board[row][col] != 0) {
```

```
            continue;
        }

        int candidateCount = 0;
        unsigned int candidateMask = 0;

        for (int digit = 1; digit <= 9; digit++) {
            if (isSafe(row, col, digit)) {
                candidateCount++;
                candidateMask |= (1U << digit);
            }
        }

        if (candidateCount == 0) {
            bestRow = row;
            bestCol = col;
            bestMask = 0;
            return true;
        }

        if (candidateCount < fewestCandidates) {
            fewestCandidates = candidateCount;
            bestRow = row;
            bestCol = col;
            bestMask = candidateMask;

            // MODIFIED: Continue checking all empty cells
            // to ensure no cell has zero possible candidates.
        }
    }
```

```
    }

    return bestRow != -1;
}

void solveRecursive(ofstream& outputFile) {
    int emptyRow = -1;
    int emptyCol = -1;
    unsigned int candidates = 0;

    if (!findBestEmptyCell(emptyRow, emptyCol, candidates)) {
        solutionCount++;
        cout << "\nSolution " << solutionCount << ":" << endl;
        printBoard();
        writeBoardToFile(outputFile);
        return;
    }

    if (candidates == 0) {
        return;
    }

    for (int digit = 1; digit <= 9; digit++) {
        if ((candidates & (1U << digit)) == 0) {
            continue;
        }

        board[emptyRow][emptyCol] = digit;
        solveRecursive(outputFile);
        board[emptyRow][emptyCol] = 0;
```

```
        }
    }

public:
    SudokuSolver() : solutionCount(0) {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                board[row][col] = 0;
            }
        }
    }

    bool loadPuzzle(const string& filename) {
        ifstream inputFile(filename);

        if (!inputFile.is_open()) {
            cerr << "Error: Could not open " << filename << endl;
            return false;
        }

        vector<char> cells;
        char ch;

        while (inputFile.get(ch)) {
            if ((ch >= '1' && ch <= '9') || ch == '_') {
                cells.push_back(ch);
            } else if (ch == '0') {
                cells.push_back('_');
            } else if (!isspace(static_cast<unsigned char>(ch))) {
                cerr << "Error: Invalid character in " << filename << endl;
```

```
            return false;
        }
    }

    if (cells.size() != 81) {
        cerr << "Error: " << filename
             << " must contain exactly 81 Sudoku cells." << endl;
        return false;
    }

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            char cell = cells[row * 9 + col];

            if (cell == '_') {
                board[row][col] = 0;
            } else {
                board[row][col] = cell - '0';
            }
        }
    }

    return true;
}

bool isInitialBoardValid() {
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            if (board[row][col] == 0) {
                continue;
```

```
            }

            int digit = board[row][col];
            board[row][col] = 0;
            bool valid = isSafe(row, col, digit);
            board[row][col] = digit;

            if (!valid) {
                return false;
            }
        }
    }

    return true;
}

bool solve(const string& filename, ofstream& outputFile) {
    cout << "\n========================================" << endl;
    cout << "Puzzle file: " << filename << endl;
    cout << "========================================" << endl;
    cout << "\nOriginal puzzle:" << endl;
    printBoard();

    outputFile << "Puzzle: " << filename << "\n";

    if (!isInitialBoardValid()) {
        cout << "\nNo solution found: the original puzzle has conflicting digits." << endl;
        outputFile << "No solution found.\n";
```

```
        // MODIFIED: Check whether writing succeeded.
        return outputFile.good();
    }

    solutionCount = 0;

    solveRecursive(outputFile);

    if (solutionCount == 0) {
        cout << "\nNo solution found." << endl;
        outputFile << "No solution found.\n";
    } else {
        cout << "\nTotal solutions found: " << solutionCount << endl;
        outputFile << "Solutions found: " << solutionCount << "\n";
    }

    // MODIFIED: Check whether writing succeeded.
    return outputFile.good();
  }
};

int main() {
  const string filenames[] = {
    "puzzle1.txt",
    "puzzle2.txt",
    "puzzle3.txt",
    "puzzle4.txt",
    "puzzle5.txt"
  };
```

```
bool allFilesProcessed = true;

// Open one shared output file for all five puzzles.
ofstream outFile("solutions.txt");

if (!outFile.is_open()) {
    cerr << "Error: Could not create solutions.txt" << endl;
    return 1;
}

for (int i = 0; i < 5; i++) {
    const string& filename = filenames[i];
    SudokuSolver solver;

    if (solver.loadPuzzle(filename)) {
        if (!solver.solve(filename, outFile)) {
            allFilesProcessed = false;
        }
    } else {
        outFile << "Puzzle: " << filename
                << "\nNo solution found.\n";

        allFilesProcessed = false;
    }
}

outFile.close();

// Check for errors that occurred while closing/flushing the file.
```

```
    if (!outFile) {
        cerr << "Error: Failed to finish writing solutions.txt" << endl;
        allFilesProcessed = false;
    }

    return allFilesProcessed ? 0 : 1;
}
```